



Feed-forward Neural Networks

Lecture 2.1: Neurons and Feed Forward Networks





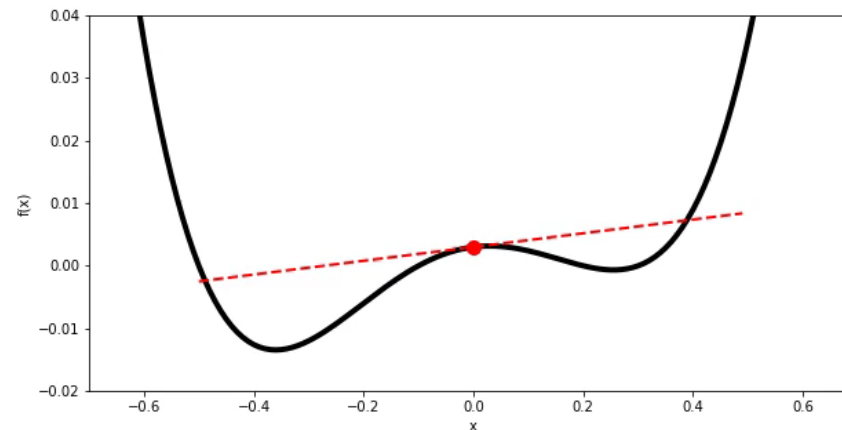
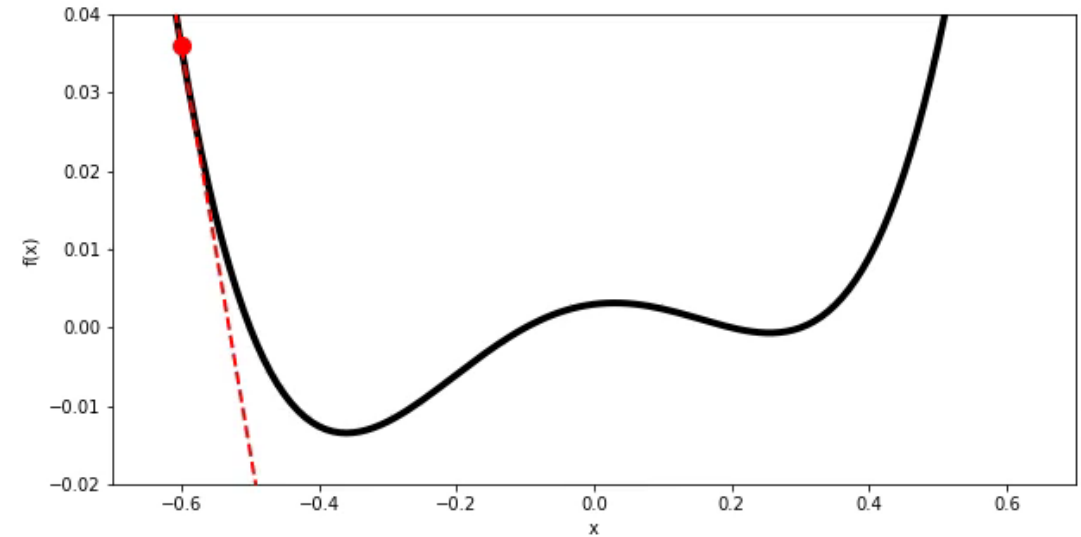
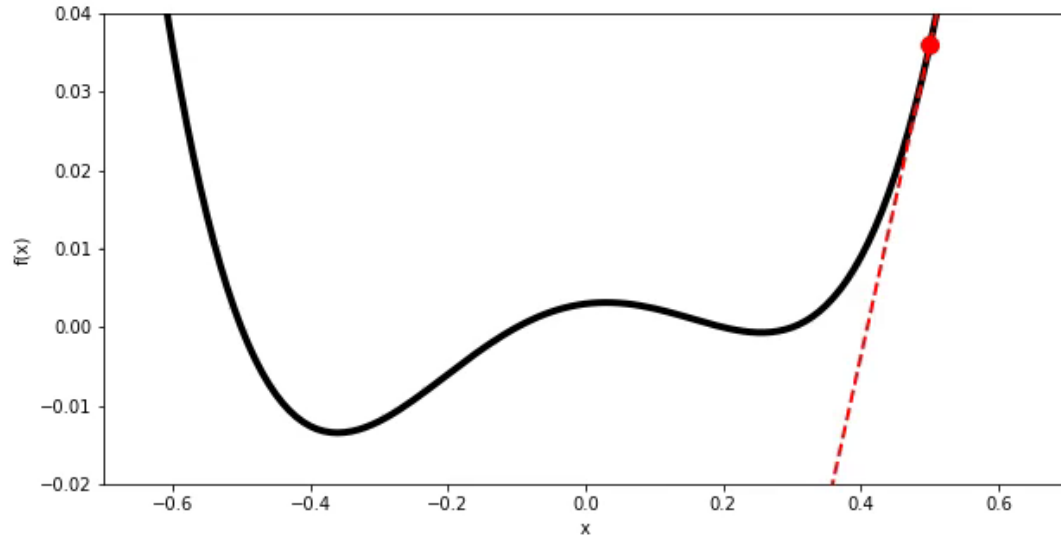
RECAP

From Last Lecture

Gradient descent update rule:

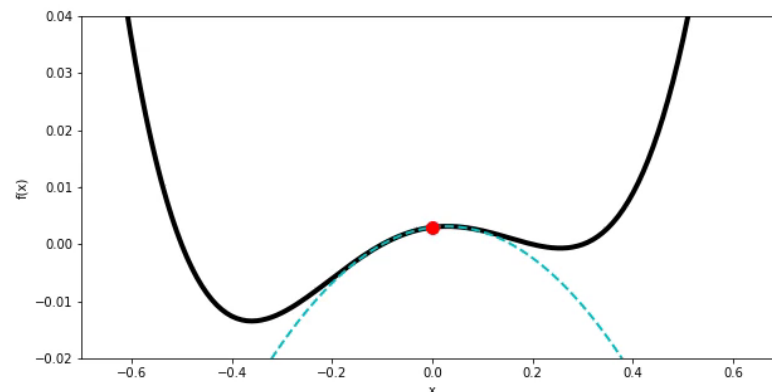
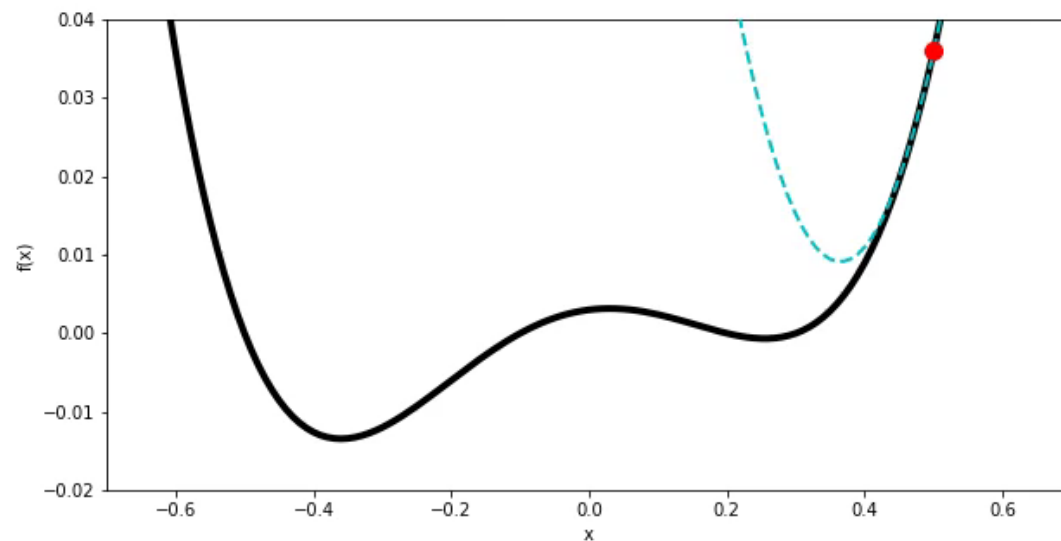
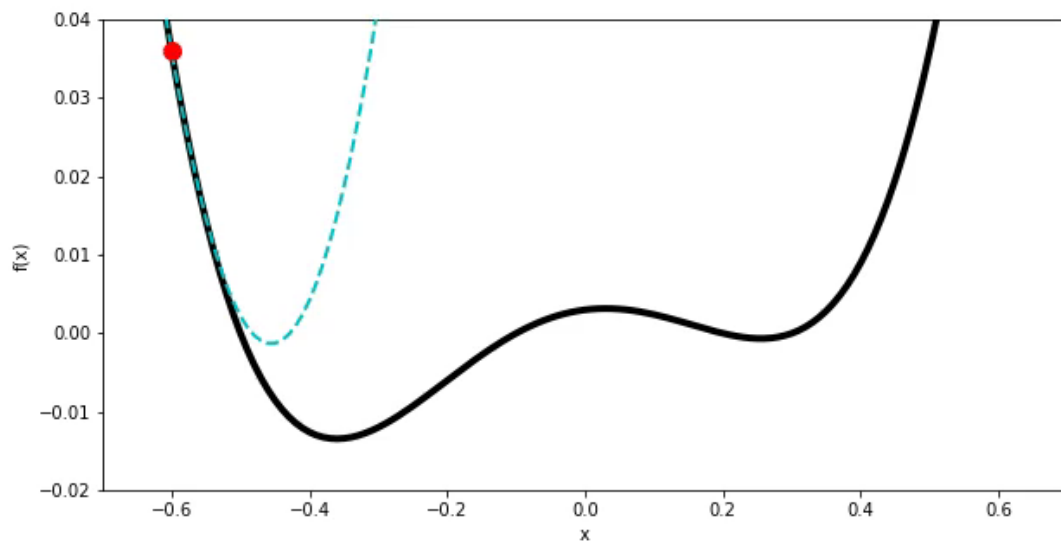
$$x^{(t+1)} = x^{(t)} - \alpha \nabla f(x^{(t)})$$

$$\alpha > 0$$



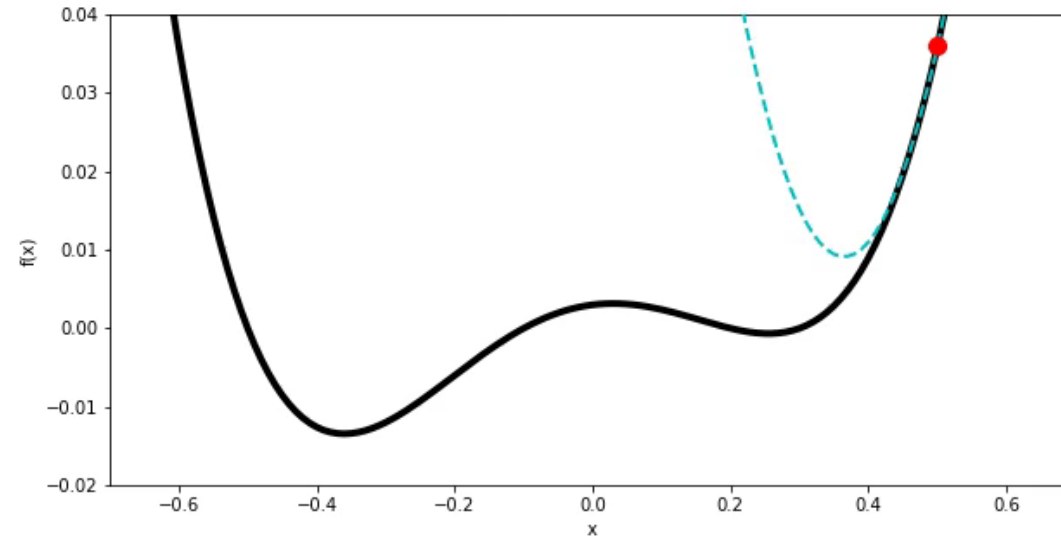
Newton Method update rule:

$$x^{t+1} = x^t - H_f(x^t)^{-1} \nabla f(x^t)$$



Newton method finds roots of the gradient – not necessarily minima

Why do you not want this loss function?



Even if deep learning can often be non-convex, we should NEVER make its job harder by giving it a loss function with local minimum!

Gradient = **0** kills any learning/optimization algorithm
Chain rule is not going to rescue you

Today's Learning Objectives

Be able to answer:

- What is a neural network?
 - What is a fully-connected layer?
 - What are common output mechanisms?
- What can a neural network represent?
- What are the intuitions behind backpropagation?
- What is forward / reverse mode differentiation?





Where it all starts: Maximum likelihood Estimation (MLE)

- The BIG assumptions of machine learning:
 - Training examples are **independent** and **identically distributed** (i.i.d.)
 - Assume that $p(\mathbf{x})$ is not modeled
 - Suppose we observed a lot of data $\{\mathbf{X}, \mathbf{Y}\} = \{(\mathbf{x}_1, \mathbf{y}_1), (\mathbf{x}_2, \mathbf{y}_2), \dots, (\mathbf{x}_n, \mathbf{y}_n)\}$. Our goal is to “maximize the likelihood” that we observed this data: $\max p(\mathbf{X}, \mathbf{Y})$
 - Hence, we make the decompositions

$$\begin{aligned} \max p(\mathbf{X}, \mathbf{Y}) &\xLeftrightarrow{\text{Use i.i.d.}} \max p(\mathbf{x}_1, \mathbf{y}_1) p(\mathbf{x}_2, \mathbf{y}_2) \dots p(\mathbf{x}_n, \mathbf{y}_n) \xLeftrightarrow{\text{no modeling } p(\mathbf{x})} \max_{\theta} p_{\theta}(\mathbf{y}_1 | \mathbf{x}_1) p_{\theta}(\mathbf{y}_2 | \mathbf{x}_2) \dots p_{\theta}(\mathbf{y}_n | \mathbf{x}_n) \\ &\xLeftrightarrow{\text{Take log}} \max_{\theta} \sum_{i=1}^n \log p_{\theta}(\mathbf{y}_i | \mathbf{x}_i) \end{aligned}$$

In classification, we should model $p_{\theta}(\mathbf{y}_i|\mathbf{x}_i)$ But it is a probability with range $[0,1]$

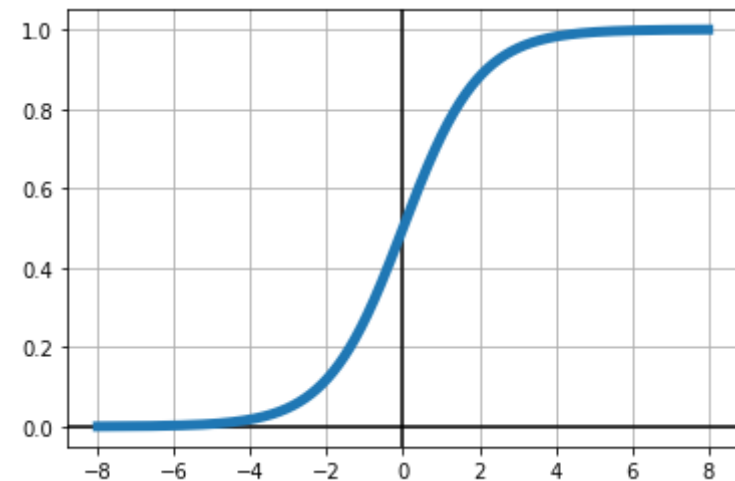
What if we get an $f_{\theta}(\mathbf{x})$ with range $(-\infty, \infty)$?

We will transform it to range $[0,1]$

Binary classification:

$$\text{Sigmoid}(a) = \frac{1}{1 + e^{-a}}$$

If we assume $f_{\theta}(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + b$, we get logistic regression



NOTE: (If we assume $\mathbf{x} = [x; 1]$ then we can remove b from the notation)



Model assumptions:

$$y_i | \mathbf{x}_i \sim \text{Bernoulli}(\theta = p_1)$$

$$\Rightarrow P(y_i | \mathbf{x}_i; \theta) = p_1^{y_i} (1 - p_1)^{(1-y_i)}$$

We then want to estimate $p_1 = P(y_i = 1 | \mathbf{x}_i) = \sigma(f_\theta(\mathbf{x}_i))$

Write out log-likelihood of the training data as a function of parameters:

$$\sum_{i=1}^N \log P(y_i | \mathbf{x}_i, \boldsymbol{\theta}) = \sum_{i=1}^N y_i \log \sigma(f_\theta(\mathbf{x}_i)) + (1 - y_i) \log(1 - \sigma(f_\theta(\mathbf{x}_i)))$$

Finding the best weights comes down to maximizing this

The Cross-Entropy Loss Function

$$\sum_{i=1}^N \log P(y_i | \mathbf{x}_i, \boldsymbol{\theta}) = \sum_{i=1}^N y_i \log \sigma(f_{\boldsymbol{\theta}}(\mathbf{x}_i)) + (1 - y_i) \log(1 - \sigma(f_{\boldsymbol{\theta}}(\mathbf{x}_i)))$$

$$= \sum_{i=1}^N -y_i \log(1 + e^{-f_{\boldsymbol{\theta}}(\mathbf{x}_i)}) - (1 - y_i) \log(1 + e^{f_{\boldsymbol{\theta}}(\mathbf{x}_i)})$$

Note $\sigma(a) = \frac{1}{1 + e^{-a}}$

Note $y_i \in \{0, 1\}$

$$= \sum_{i=1}^N -\log(1 + e^{-y_i^* f_{\boldsymbol{\theta}}(\mathbf{x}_i)}) \quad \text{where } y_i^* = 2y_i - 1 \in \{-1, 1\}$$

Same loss function as logistic regression
or XGBoost



Outputting for multi-class classification

Softmax function: It turns a real-valued vector $\mathbb{R}^d$ to a categoric distribution -- i.e. each element in $(0,1)$ and it sums to 1.

$$\text{Softmax} \left(\begin{bmatrix} z_1 \\ z_2 \\ \vdots \\ z_d \end{bmatrix} \right) = \begin{bmatrix} e^{z_1} \\ e^{z_2} \\ \vdots \\ e^{z_d} \end{bmatrix} \frac{1}{\sum_{i=1}^d e^{z_i}}$$

"**max**" - most of the probability comes from the largest x_i

"**soft**" - differentiable and still assigns non-zero probability to other entries



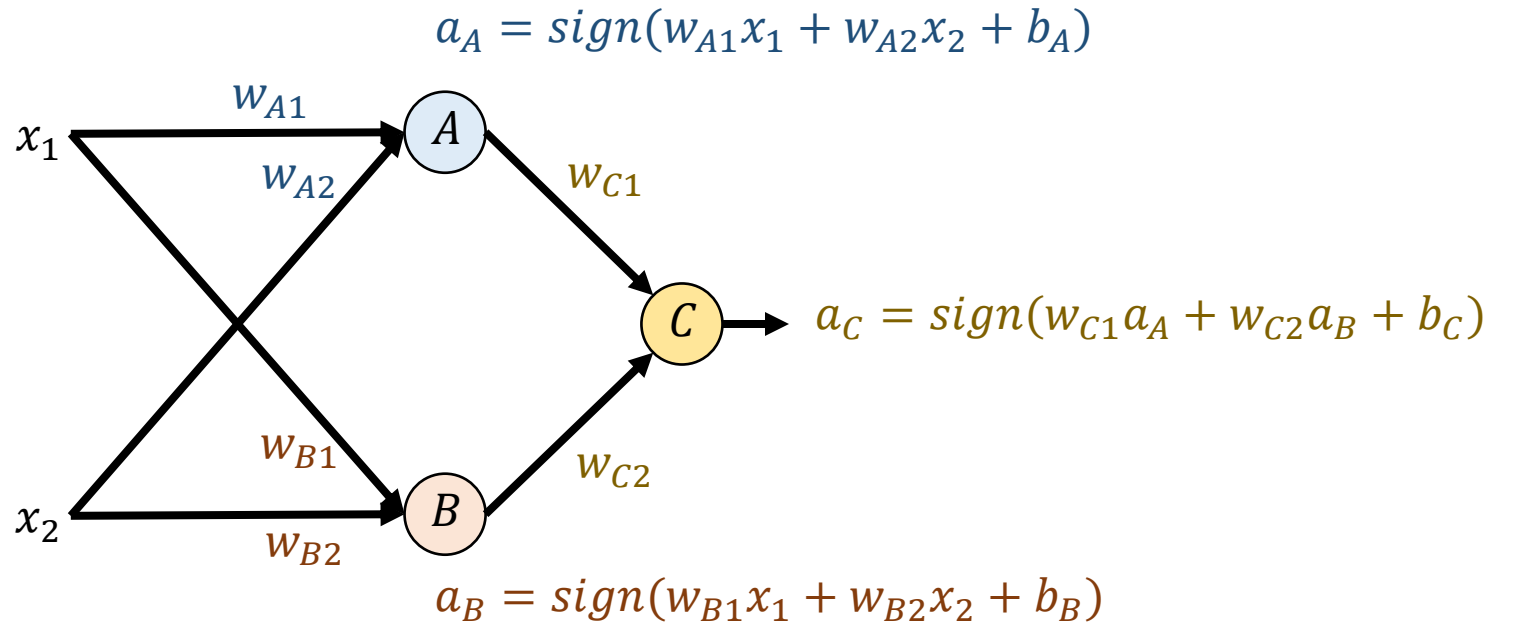
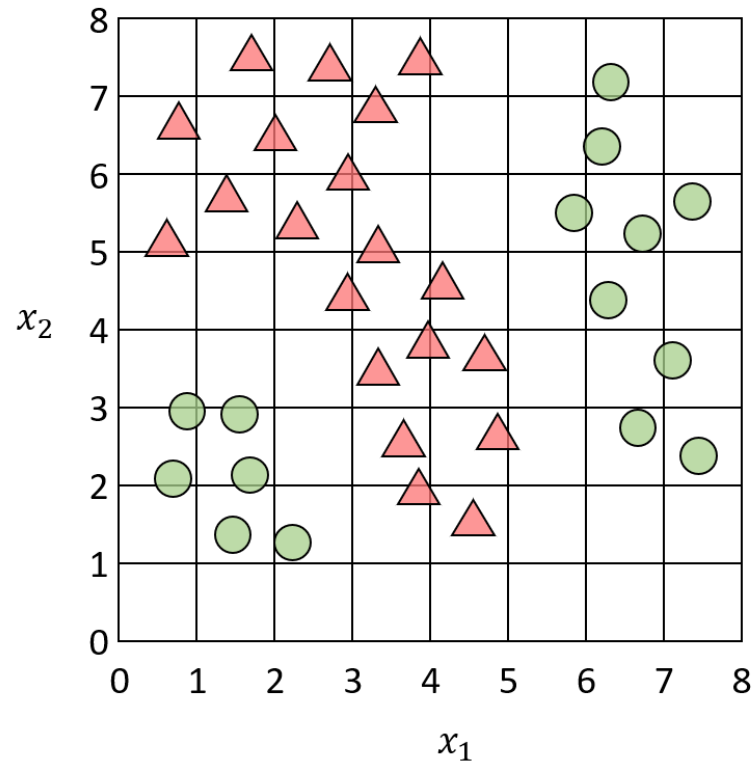
- “Soft” argmax function
- e.g. $\mathbf{x}^\top \mathbf{w}_1 = 5, \mathbf{x}^\top \mathbf{w}_2 = 3$
 - $P(y = 1|\mathbf{x}) = 88\%$
 - $P(y = 2|\mathbf{x}) = 12\%$
- e.g. $\mathbf{x}^\top \mathbf{w}_1 = 15, \mathbf{x}^\top \mathbf{w}_2 = 10$
 - $P(y = 1|\mathbf{x}) = 99.33\%$
 - $P(y = 2|\mathbf{x}) = 0.67\%$
- e.g. $\mathbf{x}^\top \mathbf{w}_1 = 15, \mathbf{x}^\top \mathbf{w}_2 = 12, \mathbf{x}^\top \mathbf{w}_3 = 13$
 - $P(y = 1|\mathbf{x}) = 84.4\%$
 - $P(y = 3|\mathbf{x}) = 11.4\%$
 - $P(y = 2|\mathbf{x}) = 4.2\%$
- With many classes, there will be a lot of outputs, but few probabilities larger than 1%

$$P(y = j|\mathbf{x}) = \frac{e^{\mathbf{x}^\top \mathbf{w}_j}}{\sum_{k=1}^K e^{\mathbf{x}^\top \mathbf{w}_k}}$$



How does multi-layer help?

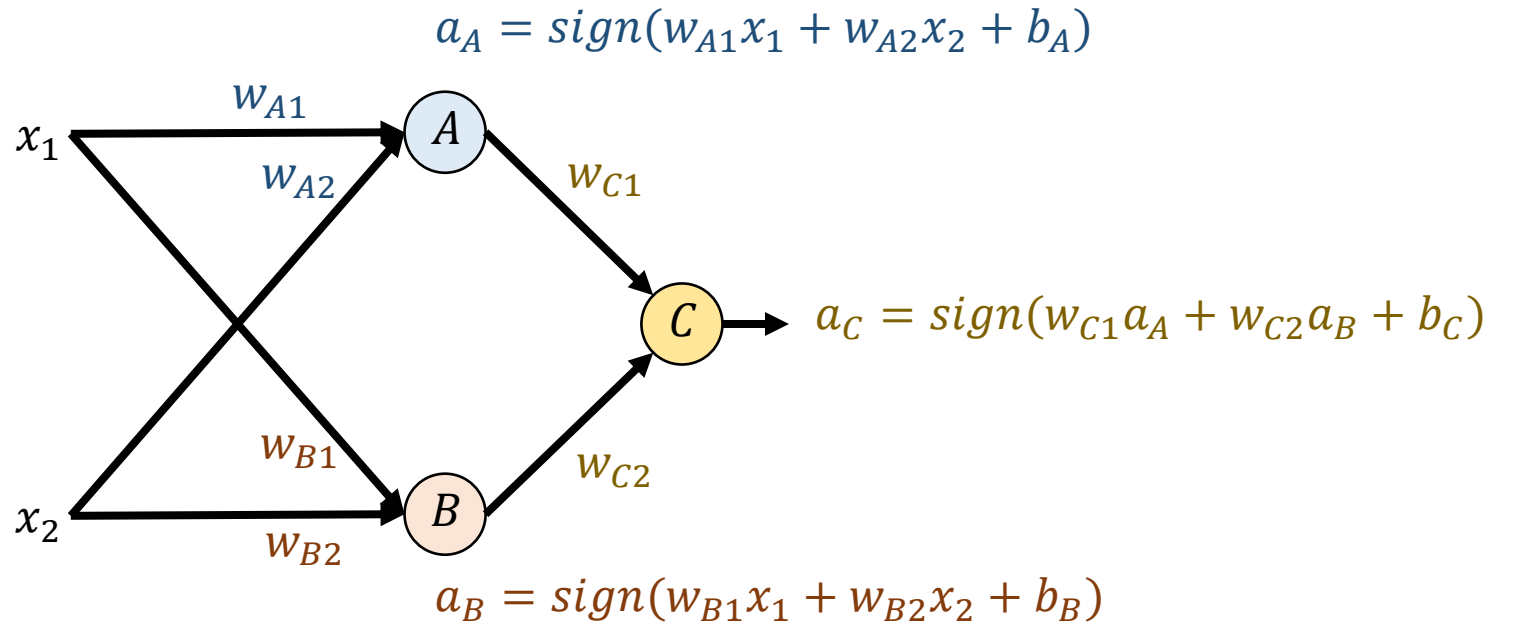
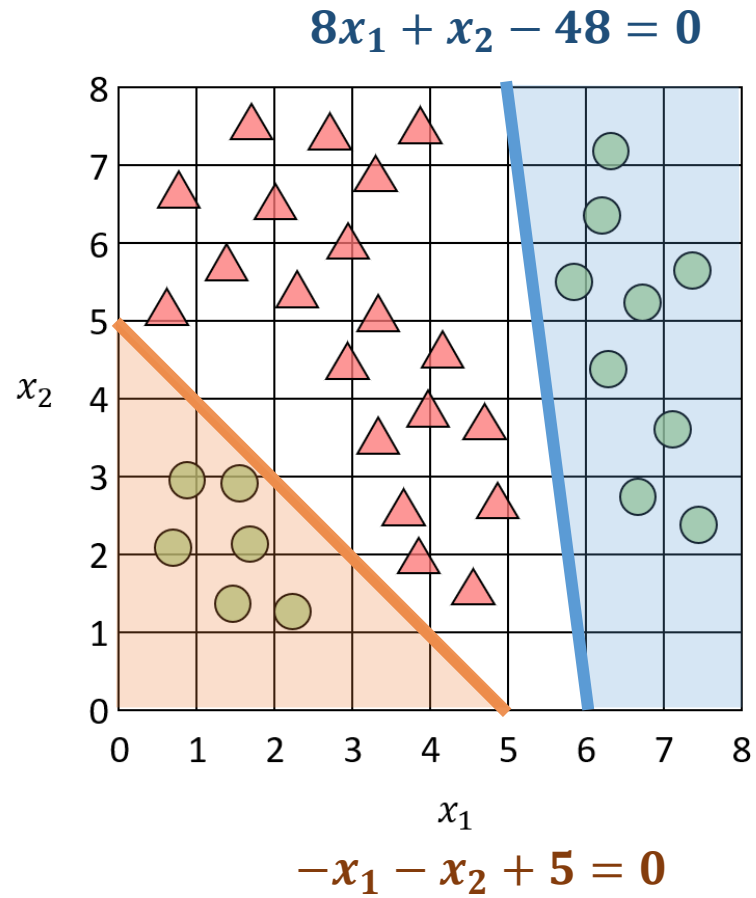
Let's consider neurons with a sign activation:



What values can we set the parameters to let the network solve this dataset?

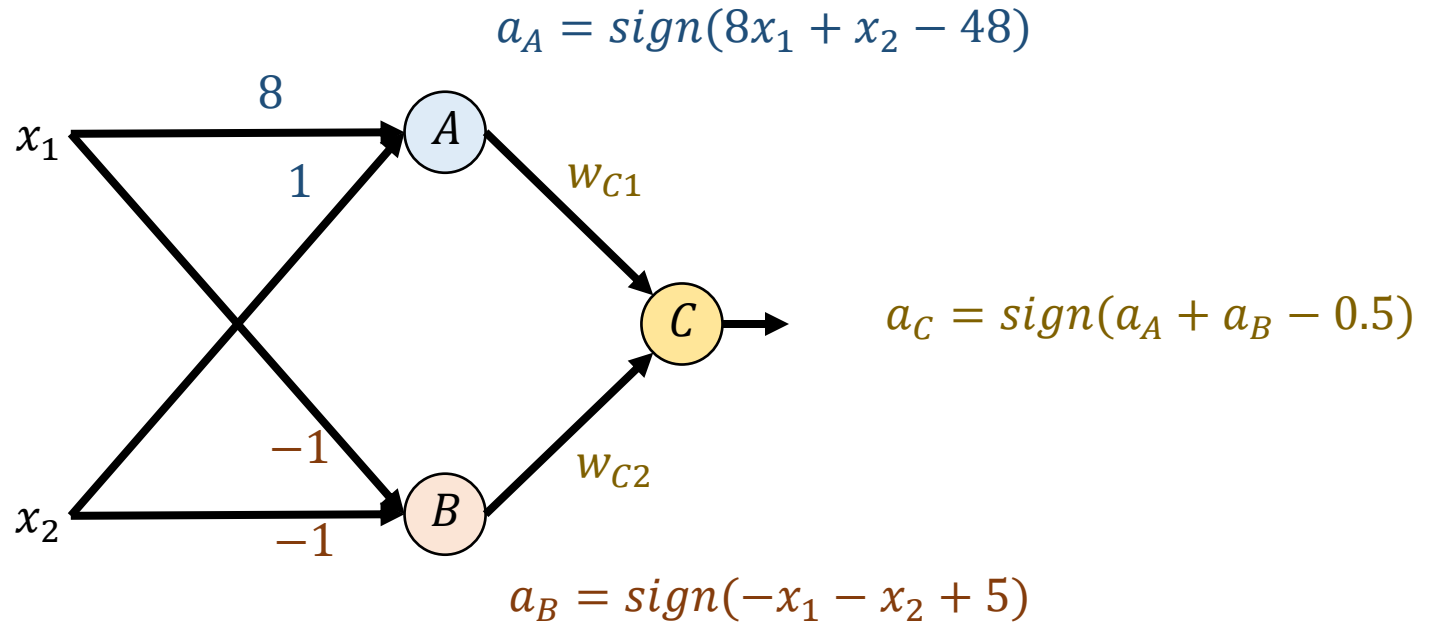
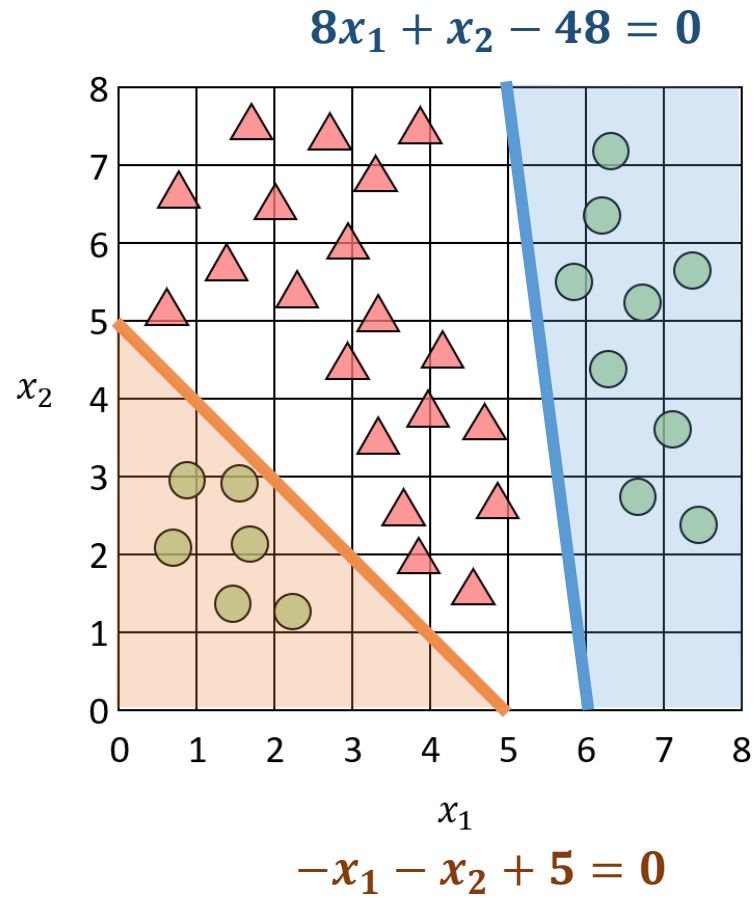


How does multi-layer help?

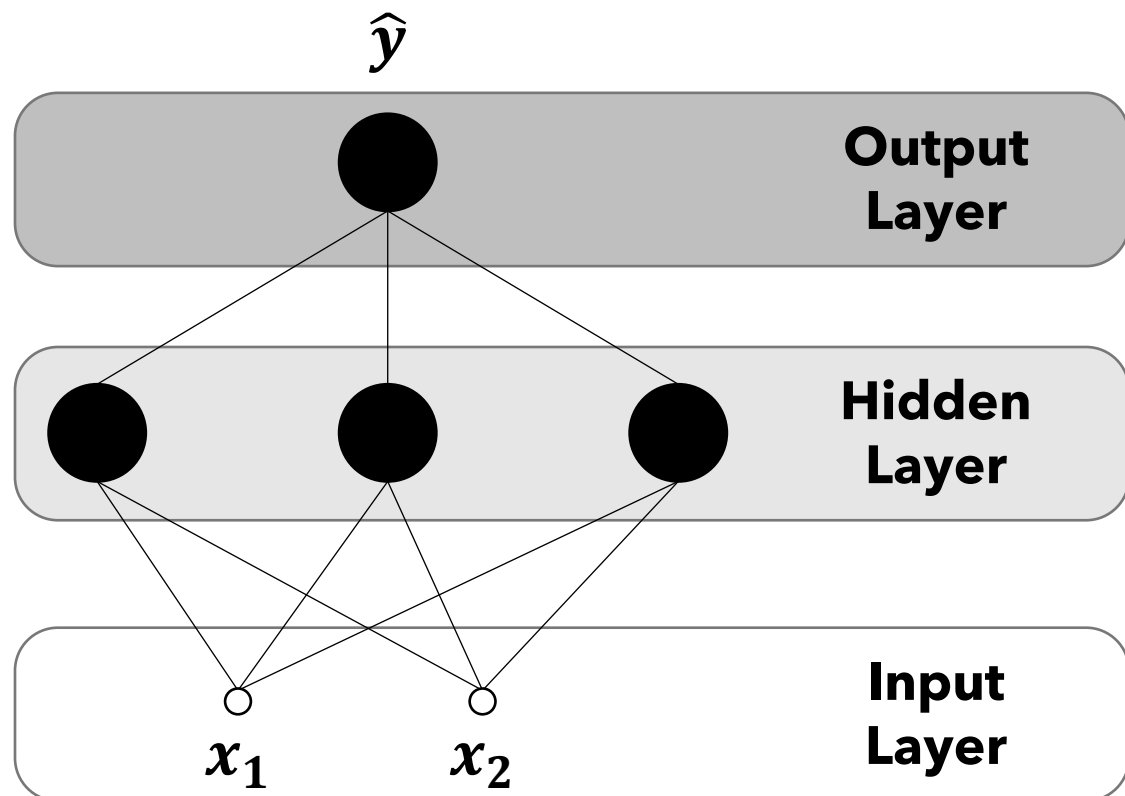




How does multi-layer help?



A Neural Network is a set of connected neurons. A very typical arrangement is a feed-forward multilayer neural network like the one shown below.



Each layer receives its input from the previous layer and forwards its output to the next – thus the **feed-forward** description.

The layers of neurons between the input and output are referred to as **hidden layers**.

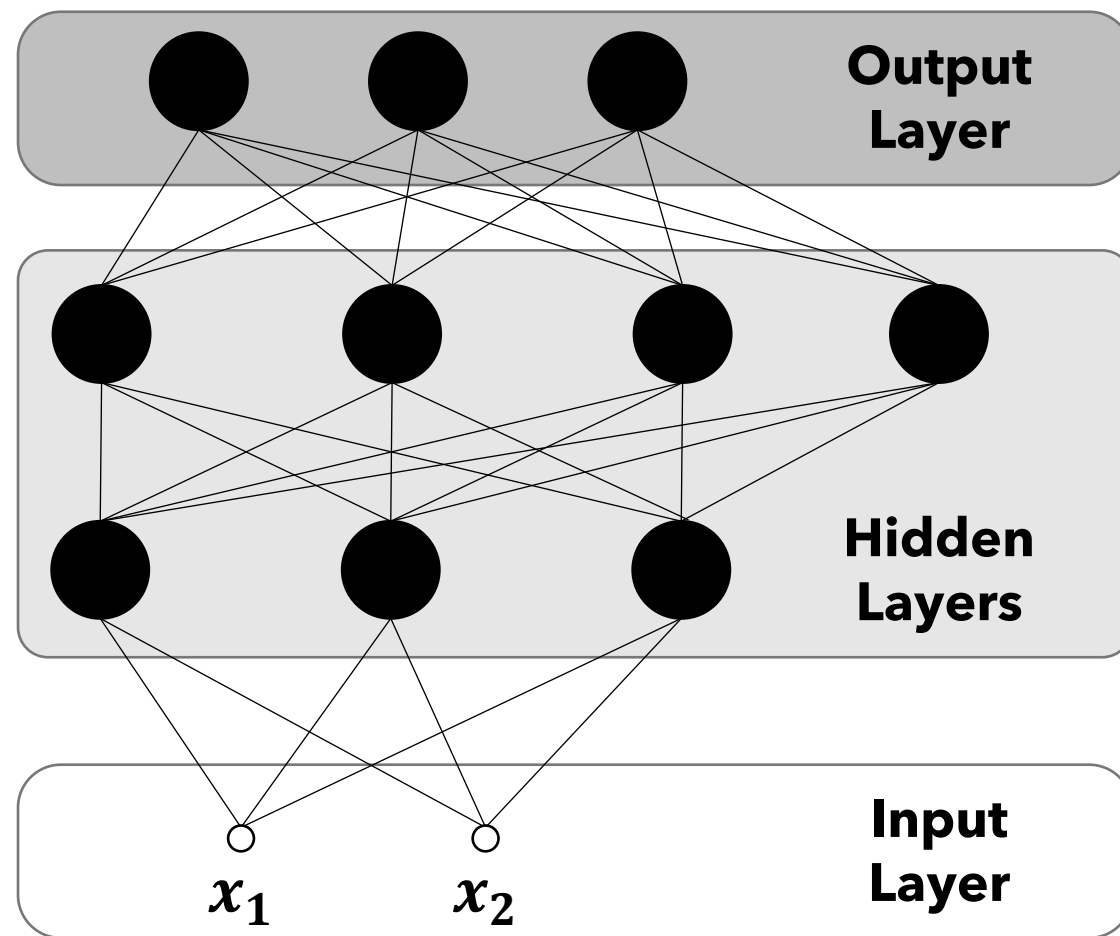
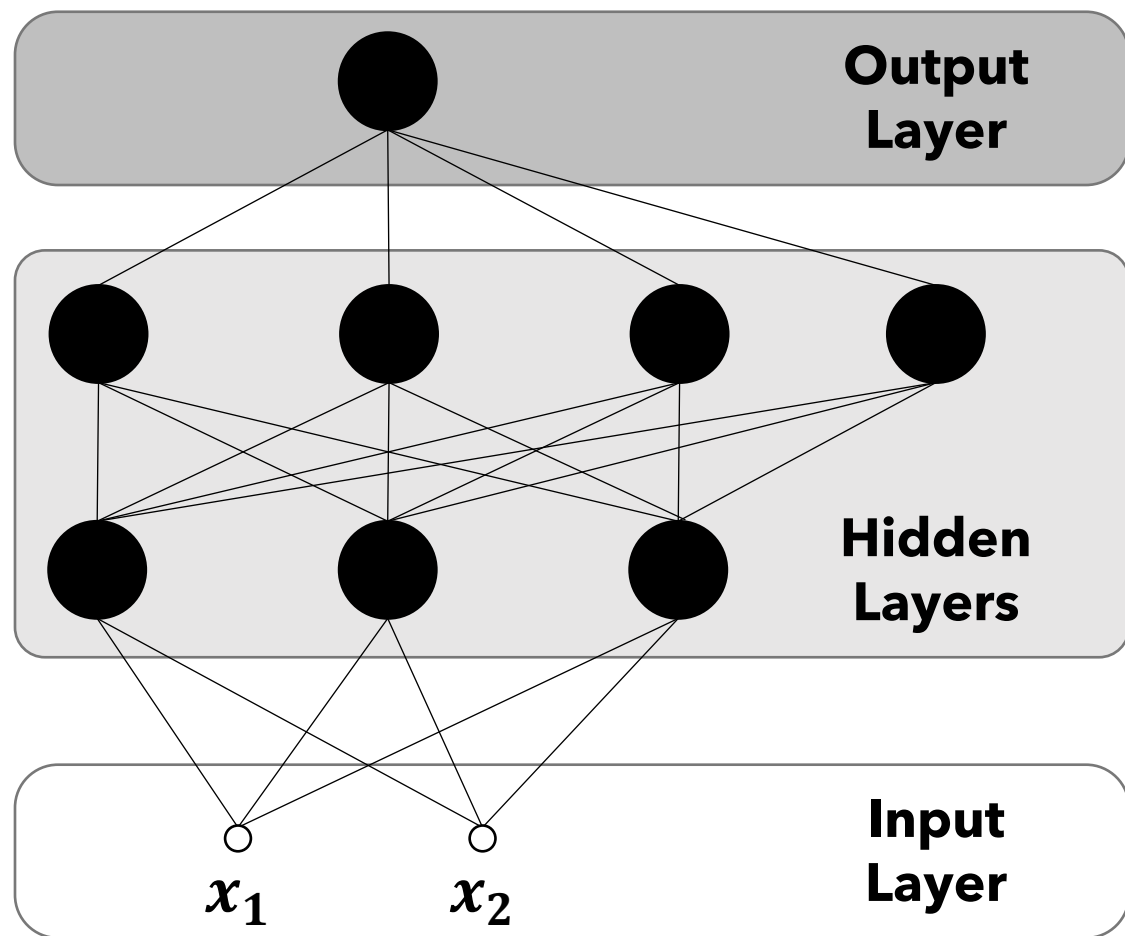
- This network for instance is a **2-layer neural** network (1 hidden and 1 output)
- Number of neurons in a layer is referred to as its width.

Activation functions in different layers can be heterogenous.

- Output layer's activation is task dependent
 - Linear for regression
 - Sigmoid or softmax for classification

Basic Multilayer Neural Network

Can make arbitrary configurations of # of hidden layers, layer widths, and number of inputs / outputs. Very flexible models!

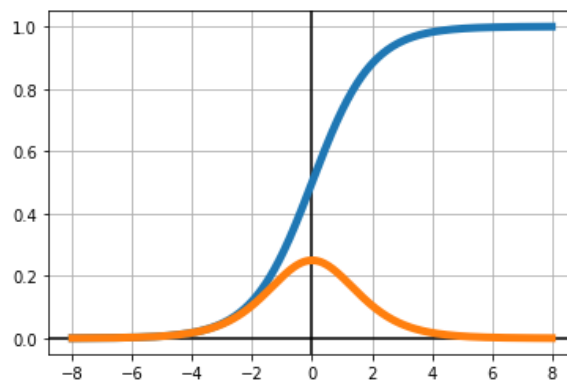




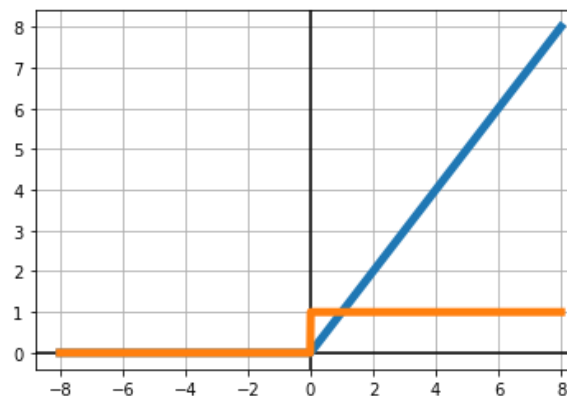
From Sigmoid to ReLU

- Sigmoid only necessary for binary classification
- In the 90s, it's also used in hidden layers
 - **No longer** the case in the 2010s

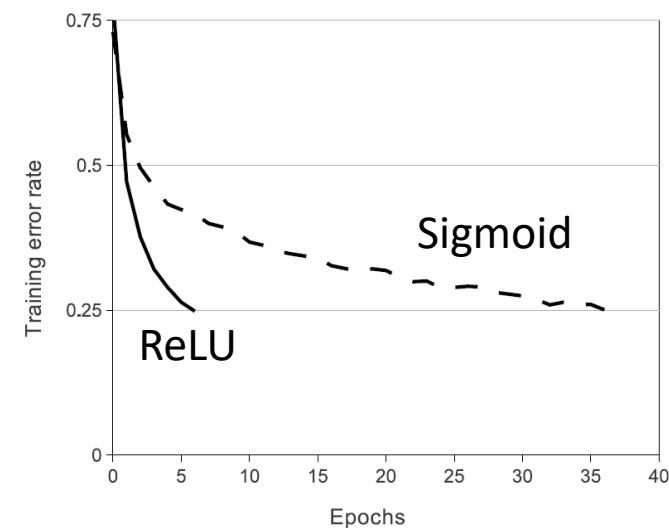
Sigmoid $\sigma(z) = \frac{1}{1+e^{-z}}$



ReLU $\sigma(z) = \max(0, z)$



In this experiment,
ReLU converged 7x
faster!



(Krizhevsky, Sutskever and Hinton, 2012)

(The ImageNet paper)

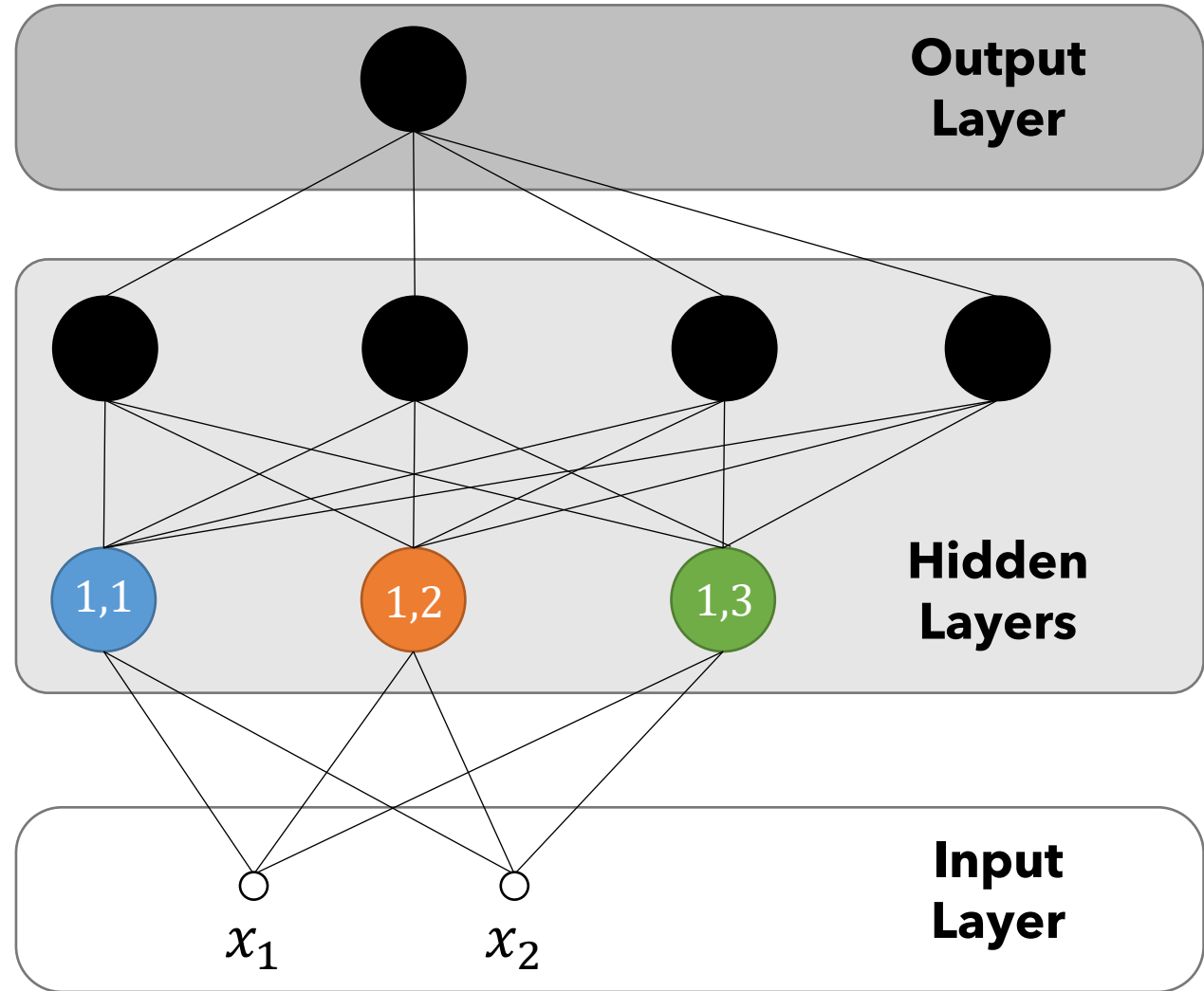


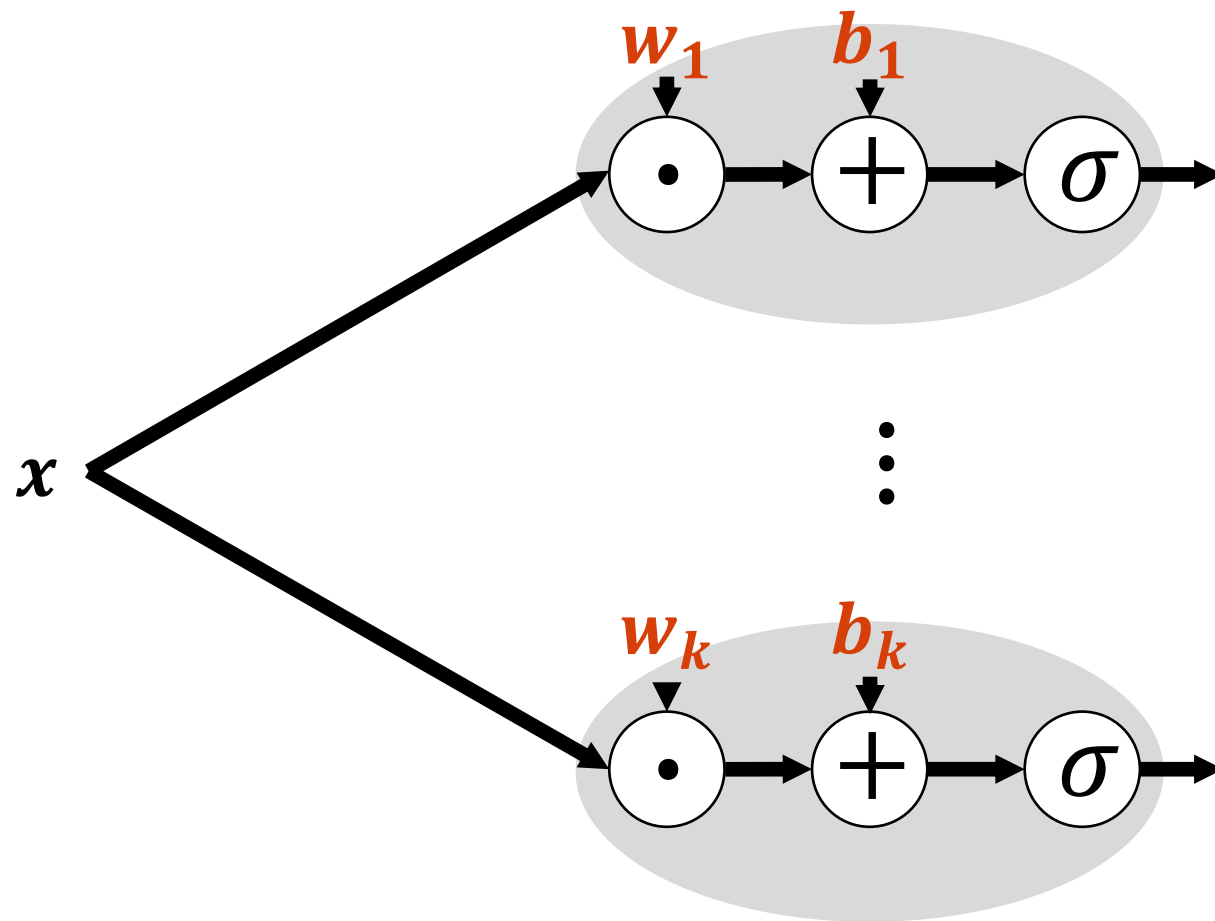
To tidy up our notation, I want to describe the output of a whole layer at once rather than write out a bunch of equations for each neuron.

$$a_{11} = \sigma(b_{11} + w_{11}^T x)$$

$$a_{12} = \sigma(b_{12} + w_{12}^T x)$$

$$a_{13} = \sigma(b_{13} + w_{13}^T x)$$





$$a_1 = \sigma(b_1 + w_1^T x)$$

$\vdots$

$$a_k = \sigma(b_k + w_k^T x)$$



$$\mathbf{a} = \sigma(\mathbf{b} + \mathbf{W}\mathbf{x})$$

$$\begin{array}{c} \mathbf{a} \\ k \times 1 \\ \left[\begin{array}{c} a^{(1)} \\ \vdots \\ a^{(k)} \end{array} \right] \end{array} = \sigma \begin{array}{c} \mathbf{b} \\ k \times 1 \\ \left[\begin{array}{c} b^{(1)} \\ \vdots \\ b^{(k)} \end{array} \right] \end{array} + \begin{array}{c} \mathbf{W} \\ k \times d \\ \left[\begin{array}{cccc} w_{11} & w_{12} & \dots & w_{1d} \\ \vdots & \vdots & \dots & \vdots \\ w_{k1} & w_{k2} & \dots & w_{kd} \end{array} \right] \end{array} \begin{array}{c} \mathbf{x} \\ d \times 1 \\ \left[\begin{array}{c} x_1 \\ \vdots \\ x_d \end{array} \right] \end{array}$$

Activation
applied
elementwise



Working on Notation for Feed-Forward Neural Networks

$$a_{11} = \sigma(b_{11} + w_{11}^T x)$$

$$a_{12} = \sigma(b_{12} + w_{12}^T x)$$

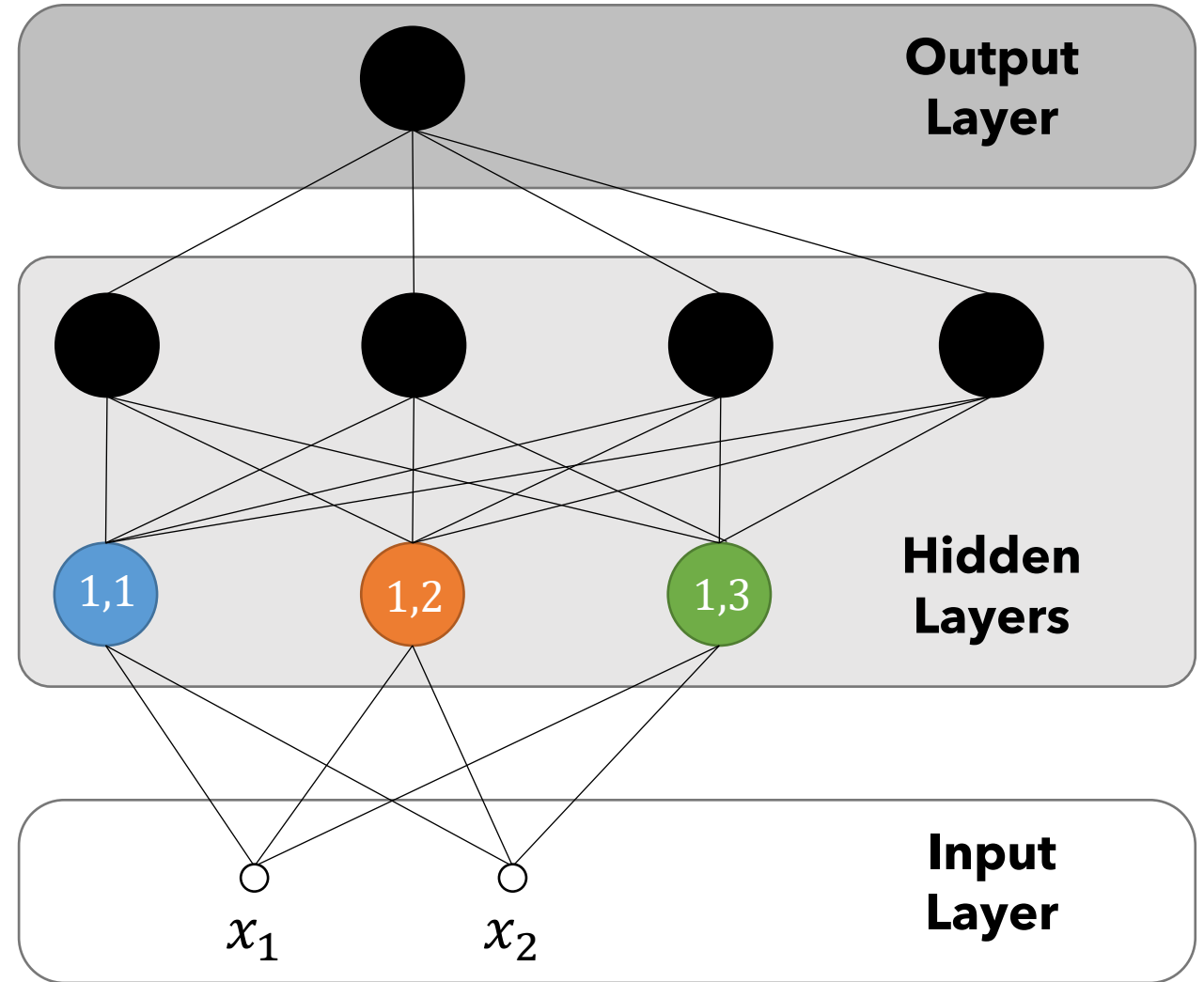
$$a_{13} = \sigma(b_{13} + w_{13}^T x)$$

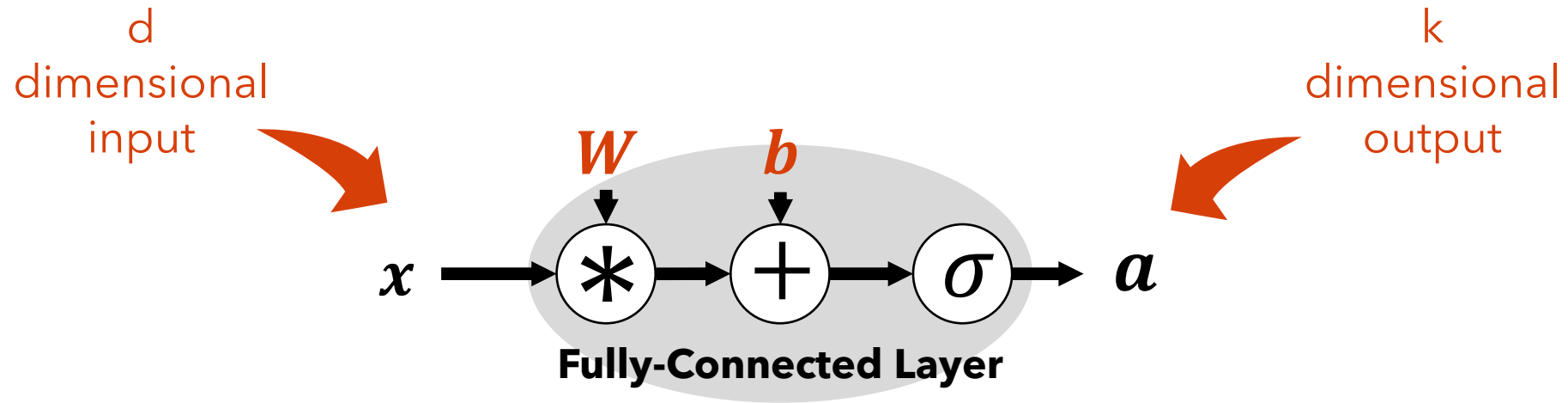


$$\begin{matrix} \mathbf{a}^{(1)} \\ 3 \times 1 \end{matrix} = \sigma \left(\begin{matrix} \mathbf{b}^{(1)} \\ 3 \times 1 \end{matrix} + \begin{matrix} \mathbf{W}^{(1)} \\ 3 \times 2 \end{matrix} \begin{matrix} \mathbf{x} \\ 2 \times 1 \end{matrix} \right)$$
$$\begin{bmatrix} a_1 \\ a_2 \\ a_3 \end{bmatrix} = \sigma \left(\begin{bmatrix} b_{11} \\ b_{12} \\ b_{13} \end{bmatrix} + \begin{bmatrix} w_{11}^T \\ w_{12}^T \\ w_{13}^T \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \right)$$



$$\mathbf{a}^{(1)} = \sigma(\mathbf{b}^{(1)} + \mathbf{W}^{(1)} \mathbf{x})$$





Hyperparameters : width of layer (k) and choice of activation functions (σ)

Learnable Parameters: $W \in \mathbb{R}^{k \times d}$ and $b \in \mathbb{R}^k$



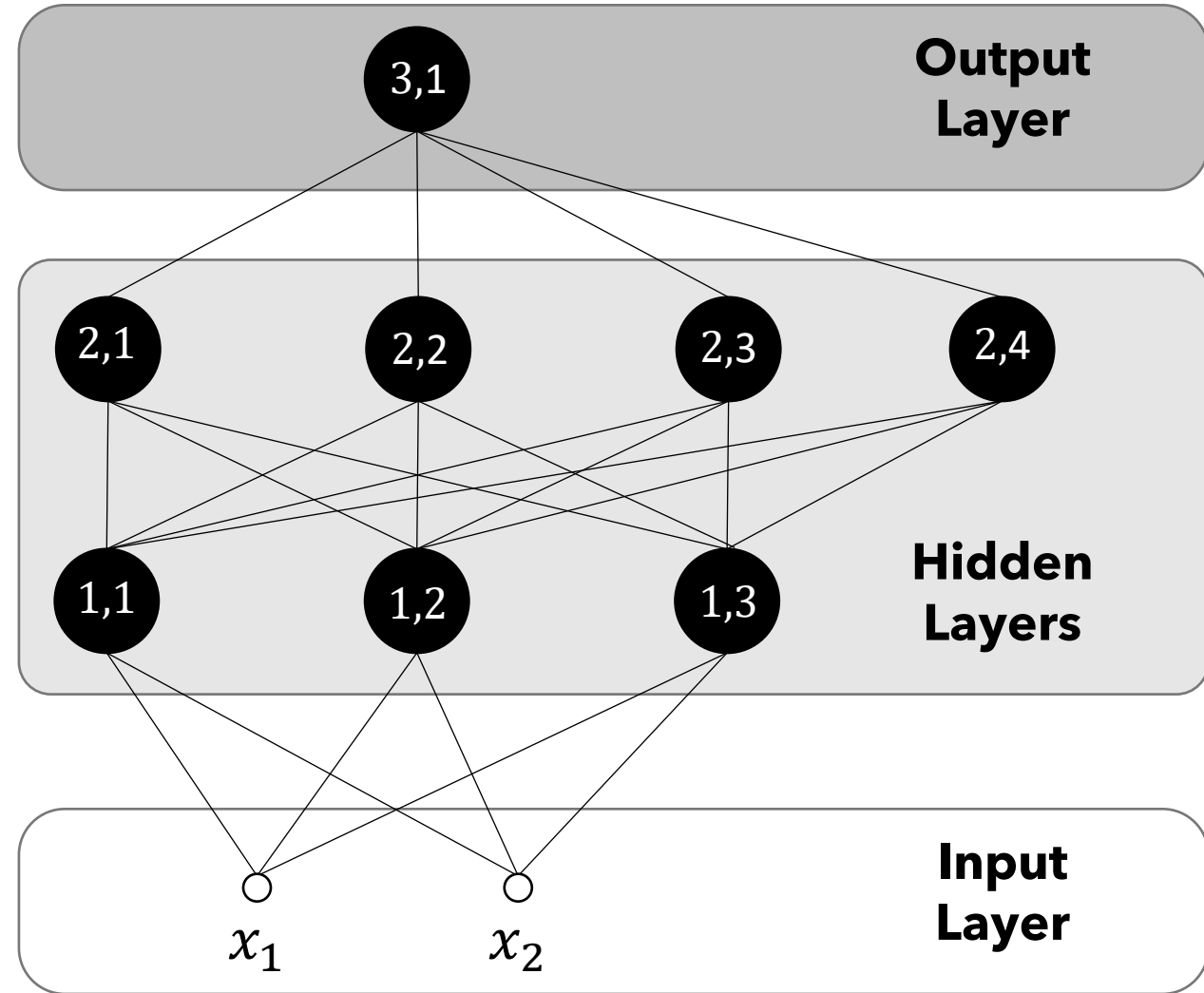
Working on Notation for Feed-Forward Neural Networks

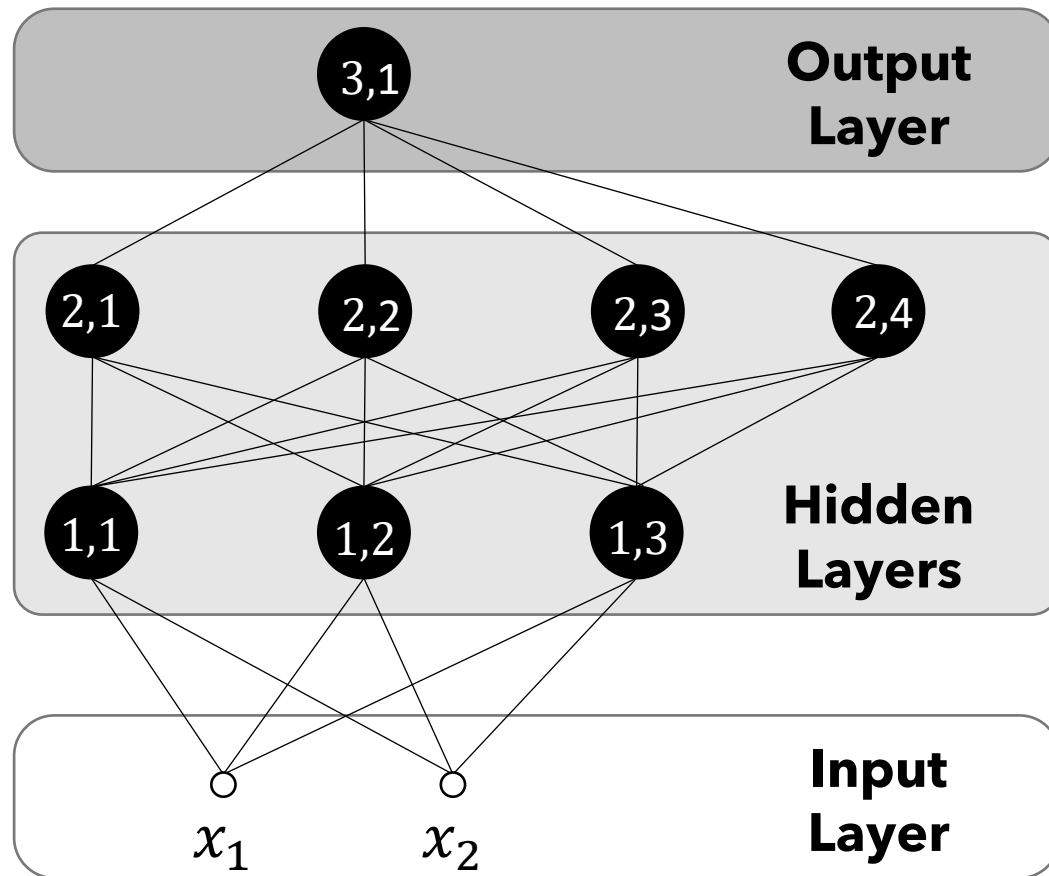
Suppose a network has L layers:

Let $W^{(1)}, W^{(2)}, \dots, W^{(L)}$ be **weight matrices** for layers $1, 2, \dots, L$.

Let $b^{(1)}, b^{(2)}, \dots, b^{(L)}$ be **bias vectors** for layers $1, 2, \dots, L$.

Let $a^{(1)}, a^{(2)}, \dots, a^{(L)}$ be **vectors of activations** for layers $1, 2, \dots, L$.



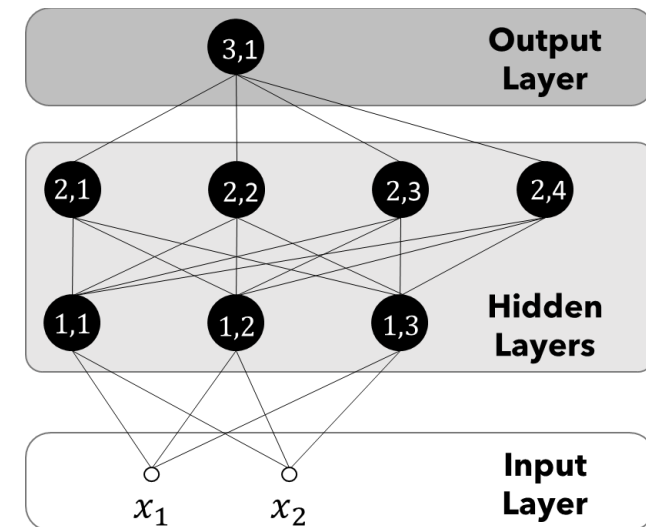
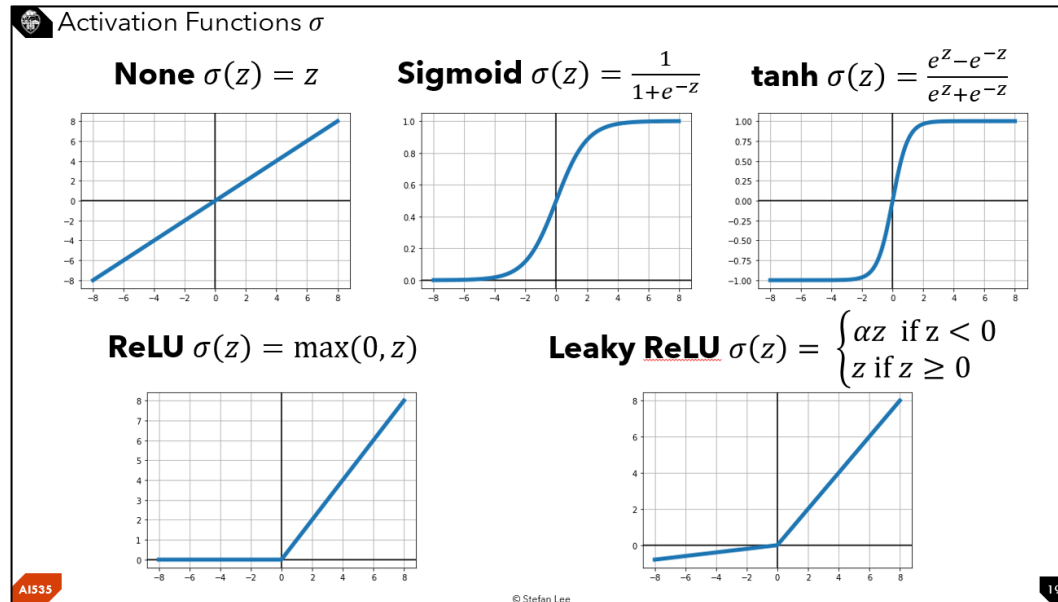


$$a = \sigma(\mathbf{b} + \mathbf{W}\mathbf{x})$$

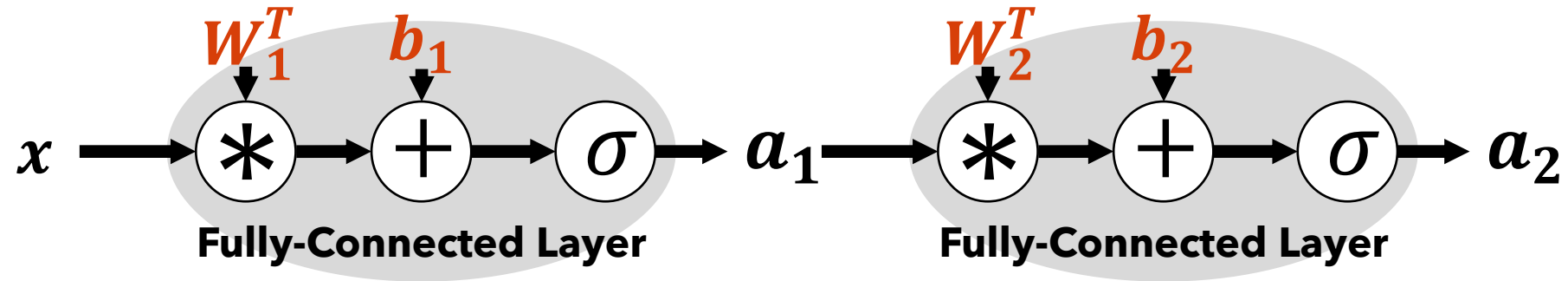
What shape is $\mathbf{W}^{(1)}$? 3×2

What shape is $\mathbf{b}^{(2)}$? 4×1

What shape is $\mathbf{a}^{(1)}$? 3×1

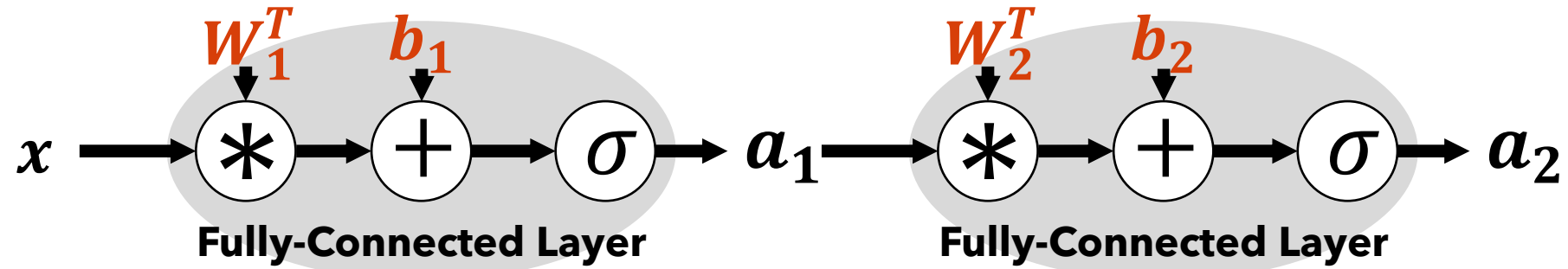


Why doesn't it make sense to use $\sigma(z) = z$ for an activation function with multiple layers?

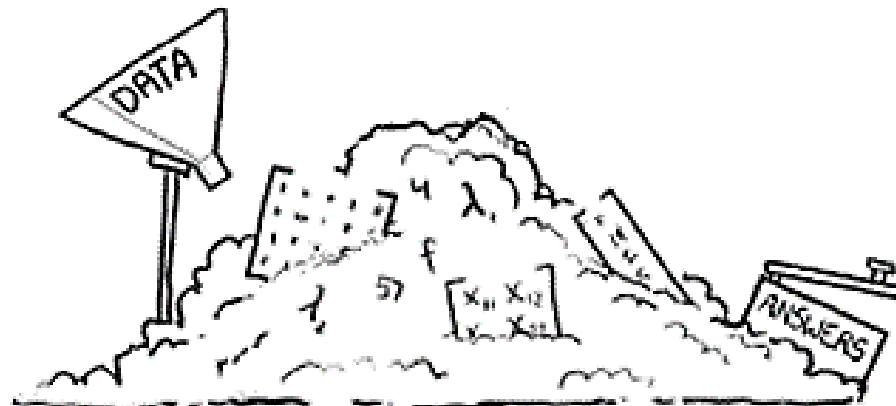


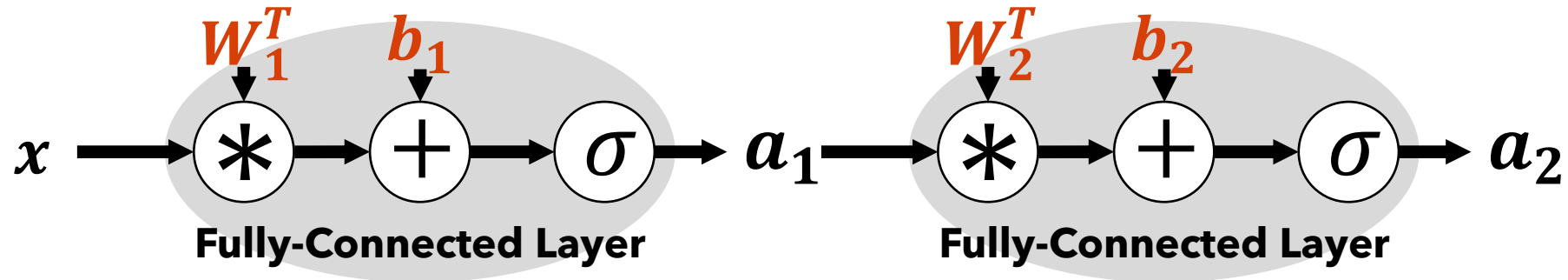
$$a_1 = \sigma(b_1 + W_1^T x)$$

$$a_2 = \sigma(b_2 + W_2^T a_1)$$



$$a_2 = \sigma(b_2 + W_2^T \sigma(b_1 + W_1^T x))$$





Hyperparameters : depth in layers (L), width of each layer ($k_1, \dots, k_L$), and choice of activation functions for each layer ($\sigma_1, \dots, \sigma_L$)

Learnable Parameters: $W_1 \in \mathbb{R}^{d \times k_1}$, $W_{i>1} \in \mathbb{R}^{k_{i-1} \times k_i}$ and $b_i \in \mathbb{R}^{k_i}$



Non-linear Function Learning: Neural networks can learn non-linear decision boundaries. But unlike the kernel methods / basis functions, we don't need to specify the exact form of this non-linear function.

Theoretically Universal Approximators: For any continuous function F over a bounded subspace of D -dimensional space, there exists a two-layer neural network $\hat{F}$ with a finite number of hidden units that approximates F arbitrarily well. That is to say, for all x in the domain of F , $|F(x) - \hat{F}(x)| < \epsilon$

- **No need for many layers:** for fully-connected networks, 1-2 hidden layers are usually enough and more layers usually only **decrease** performance

Not Deep Learning Yet: This was known in the 1990s! And neural networks are not necessarily the only/best universal approximator – SVM/Boosting/Random Forests are also universal approximators that usually work at least as well or even better than two-layer neural networks

- In this class, we **define** deep learning to be multilayer neural networks **that consider specific problem structures**. 1-hidden layer neural network on vector input x DOES NOT COUNT as deep learning (for the sake of final project)

Today's Learning Objectives

Be able to answer:

- ~~What is a neural network?~~
 - ~~What is a fully-connected layer?~~
 - ~~What are common output mechanisms?~~
- ~~What can a neural network represent?~~
- What are the intuitions behind backpropagation?
- What is forward / reverse mode differentiation?



**VOCAB**

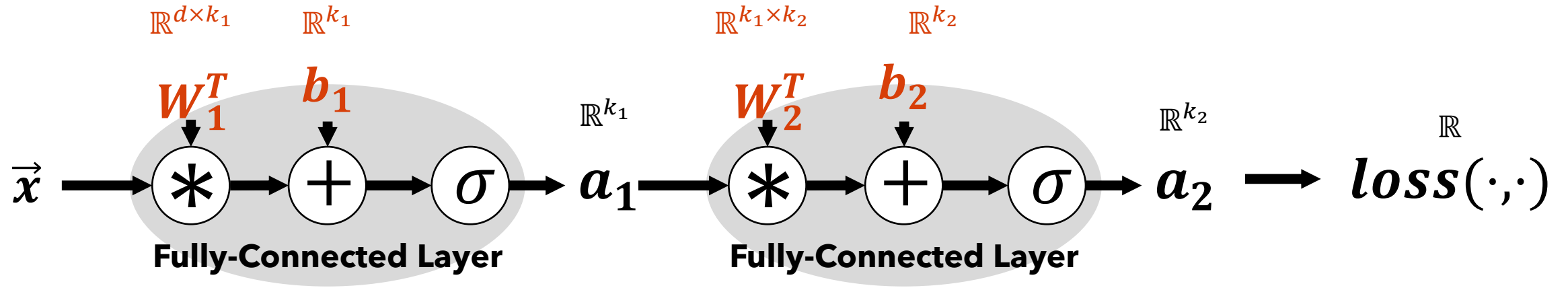
Backpropagation - A reverse-mode automatic differentiation algorithm commonly used to efficiently compute parameter gradients when training neural networks via gradient descent.

Builds off two simple observations / ideas:

- 1)** Neural networks tend to have lower dimensional outputs than inputs.
- 2)** We shouldn't recompute something we already know.



Gradients of Our Parameters

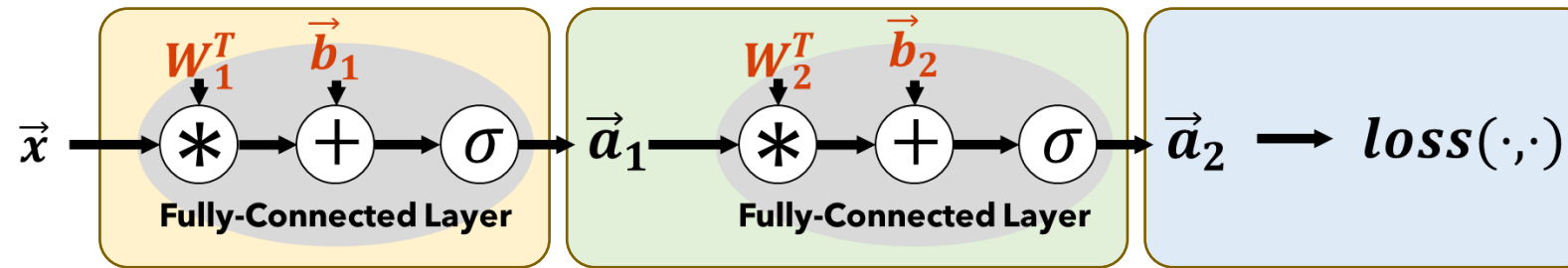


$$\frac{\delta loss}{\delta W_1} = \frac{\delta loss}{\delta a_2} \frac{\delta a_2}{\delta a_1} \frac{\delta a_1}{\delta W_1}$$

$$\text{Dim:} \quad 1 \times (dk_1) \quad 1 \times k_2 \quad k_2 \times k_1 \quad k_1 \times (dk_1)$$



Gradients of Our Parameters / The Seeds of Backprop



$$\frac{\delta L}{\delta W_2} = \boxed{\frac{\delta L}{\delta a_2}} \frac{\delta a_2}{\delta W_2}$$

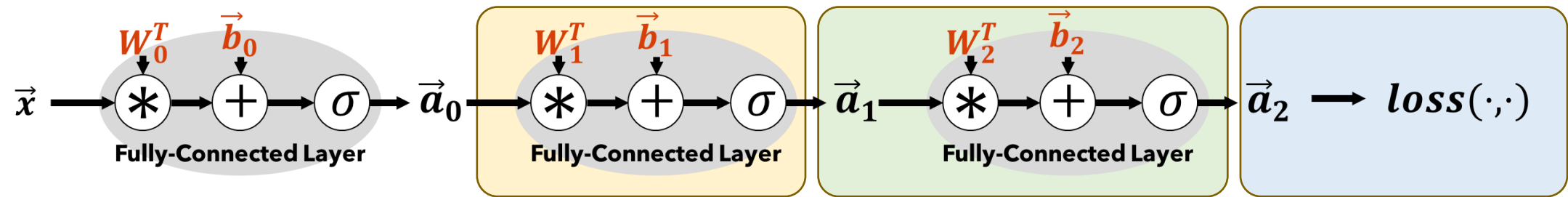
$$\frac{\delta L}{\delta b_2} = \boxed{\frac{\delta L}{\delta a_2}} \frac{\delta a_2}{\delta b_2}$$

$$\frac{\delta L}{\delta W_1} = \boxed{\frac{\delta L}{\delta a_2}} \boxed{\frac{\delta a_2}{\delta a_1}} \frac{\delta a_1}{\delta W_1}$$

$$\frac{\delta L}{\delta b_1} = \boxed{\frac{\delta L}{\delta a_2}} \boxed{\frac{\delta a_2}{\delta a_1}} \frac{\delta a_1}{\delta b_1}$$



Gradients of Our Parameters / The Seeds of Backprop



$$\frac{\delta L}{\delta \vec{W}_2} = \boxed{\frac{\delta L}{\delta \vec{a}_2}} \frac{\delta \vec{a}_2}{\delta \vec{W}_2}$$

$$\frac{\delta L}{\delta \vec{b}_2} = \boxed{\frac{\delta L}{\delta \vec{a}_2}} \frac{\delta \vec{a}_2}{\delta \vec{b}_2}$$

$$\frac{\delta L}{\delta \vec{W}_1} = \boxed{\frac{\delta L}{\delta \vec{a}_2}} \boxed{\frac{\delta \vec{a}_2}{\delta \vec{a}_1}} \frac{\delta \vec{a}_1}{\delta \vec{W}_1}$$

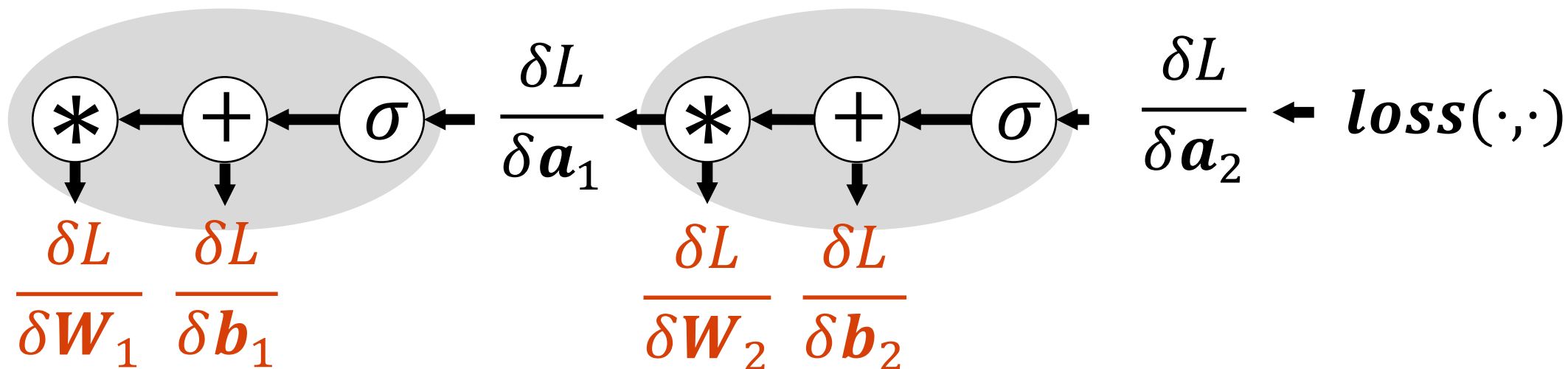
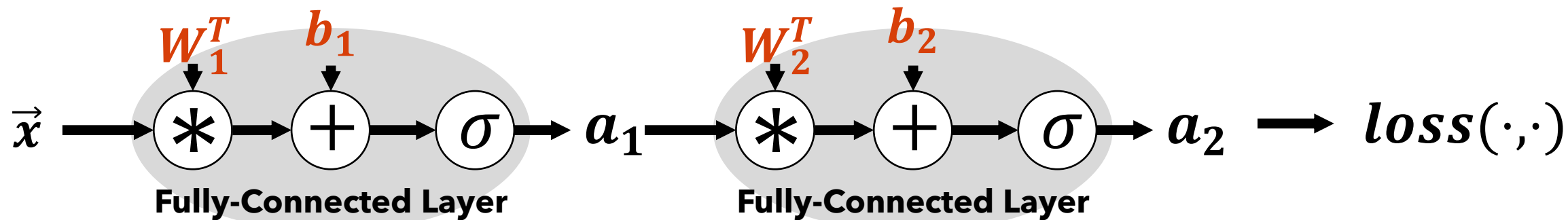
$$\frac{\delta L}{\delta \vec{b}_1} = \boxed{\frac{\delta L}{\delta \vec{a}_2}} \boxed{\frac{\delta \vec{a}_2}{\delta \vec{a}_1}} \frac{\delta \vec{a}_1}{\delta \vec{b}_1}$$

$$\frac{\delta L}{\delta \vec{W}_0} = \boxed{\frac{\delta L}{\delta \vec{a}_2}} \boxed{\frac{\delta \vec{a}_2}{\delta \vec{a}_1}} \boxed{\frac{\delta \vec{a}_1}{\delta \vec{a}_0}} \frac{\delta \vec{a}_0}{\delta \vec{W}_0}$$

$$\frac{\delta L}{\delta \vec{b}_0} = \boxed{\frac{\delta L}{\delta \vec{a}_2}} \boxed{\frac{\delta \vec{a}_2}{\delta \vec{a}_1}} \boxed{\frac{\delta \vec{a}_1}{\delta \vec{a}_0}} \frac{\delta \vec{a}_0}{\delta \vec{b}_0}$$



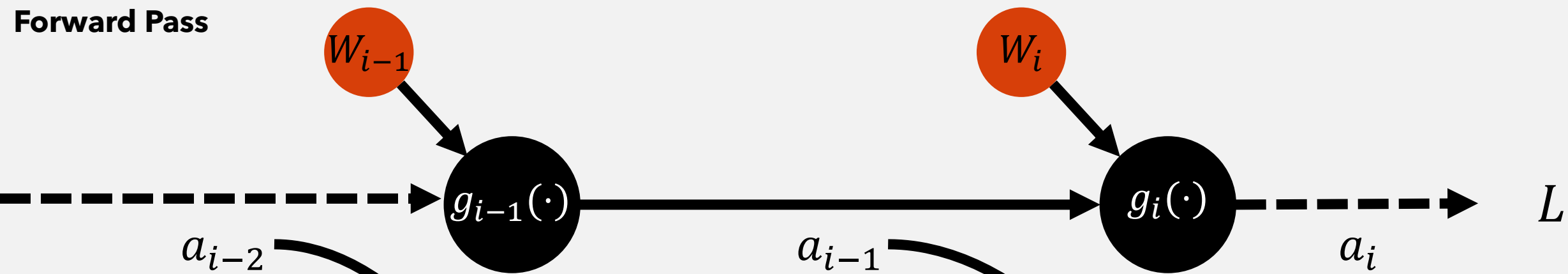
Gradients of Our Parameters / The Seeds of Backprop



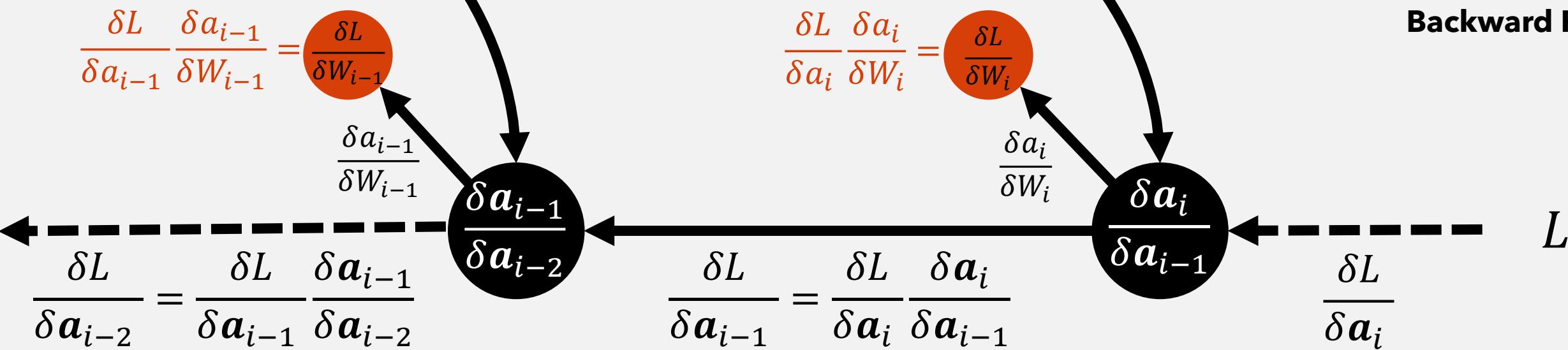


Backpropagation

Forward Pass

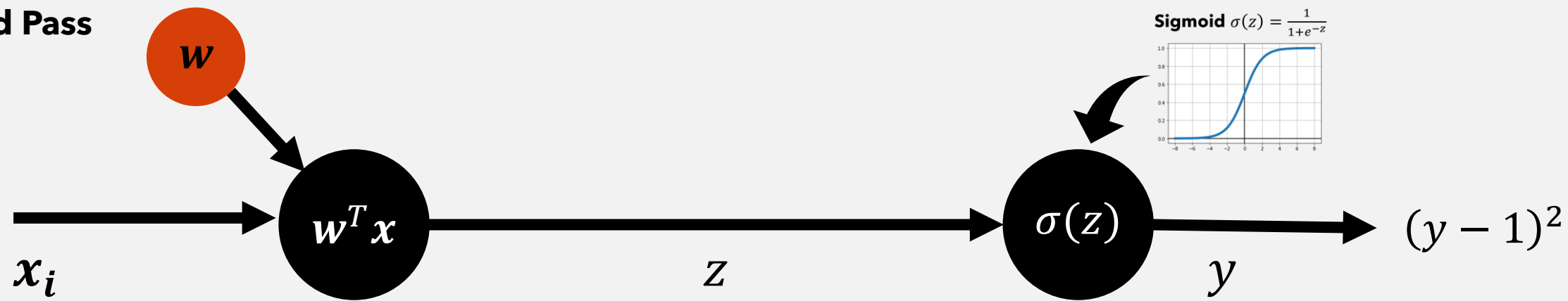


Backward Pass





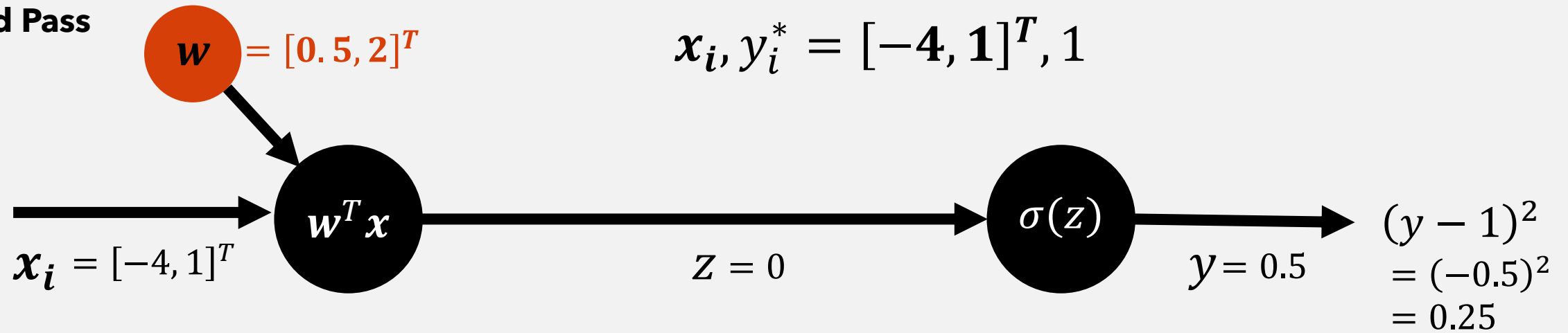
Forward Pass



Backward Pass



Forward Pass

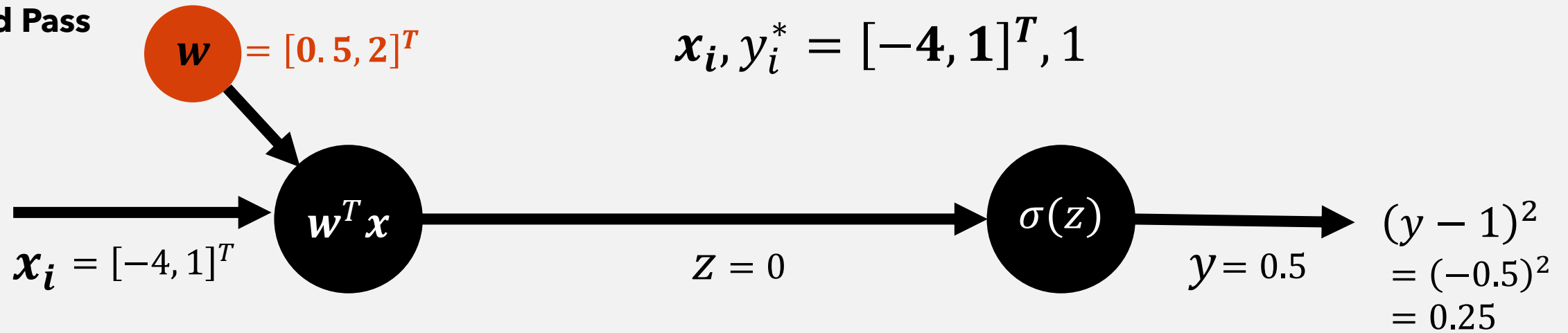


Backward Pass

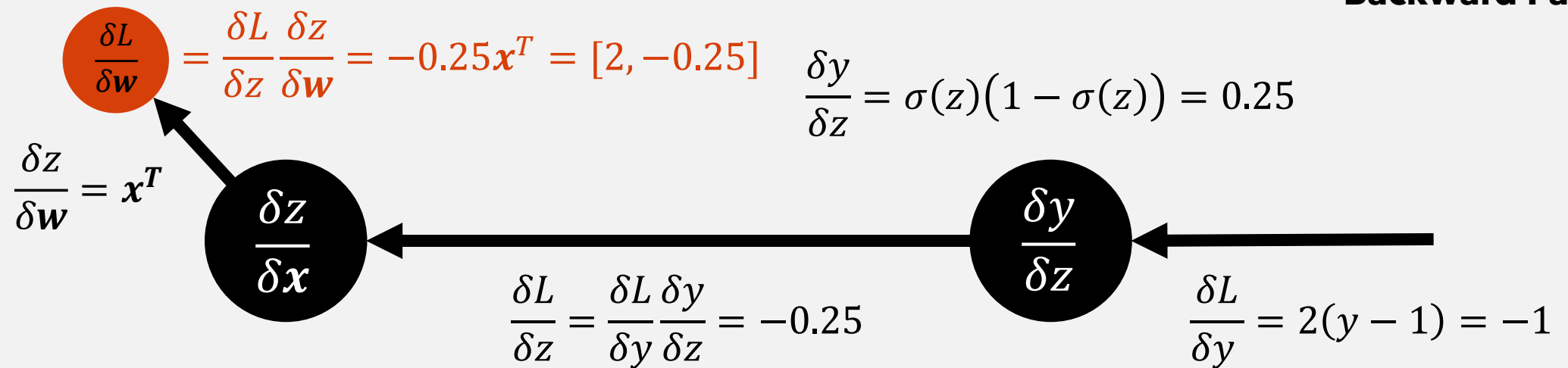


Backpropagation

Forward Pass



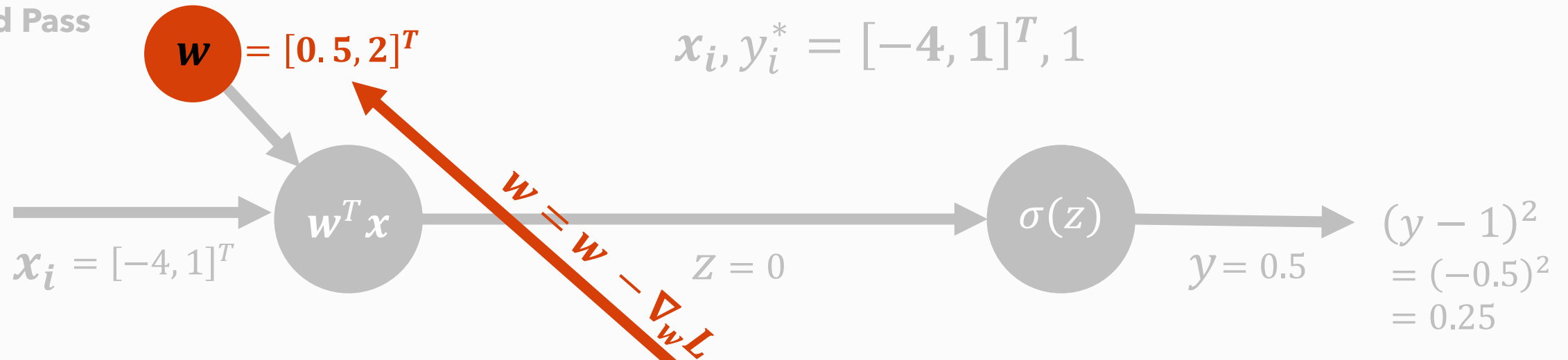
Backward Pass



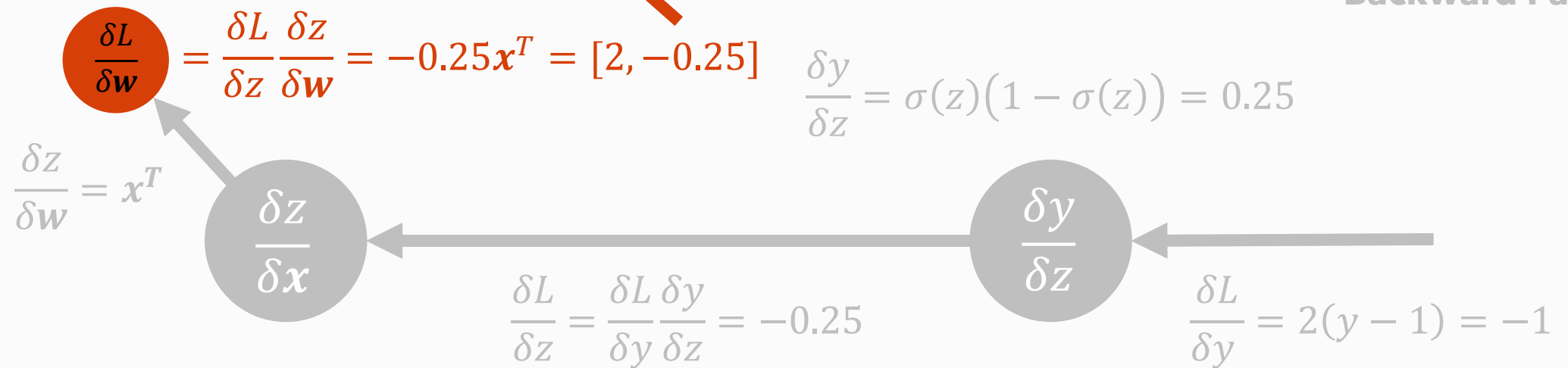


Backpropagation

Forward Pass

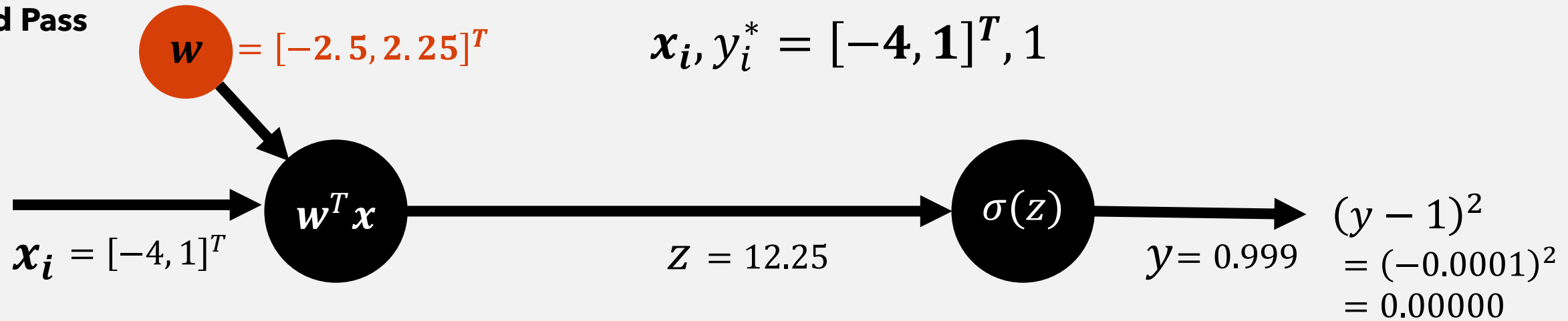


Backward Pass





Forward Pass



Backward Pass

Today's Learning Objectives

Be able to answer:

- ~~What is a neural network?~~
 - ~~What is a fully-connected layer?~~
 - ~~What are common output mechanisms?~~
- ~~What can a neural network represent?~~

- ~~What are the intuitions behind backpropagation?~~
- ~~What is forward / reverse mode differentiation?~~

